@BUNNYBETHINKING

AI Engineer Roadmap

Everything you need to go from zero to shipping real AI products — in the order it actually makes sense to learn it. No PhD required. No skipped steps.

- DO THIS FIRST
- CORE SKILL
- OPTIONAL / ADVANCED

// PRE-REQUISITE CHECK

You need *one* of these before starting: Frontend / Backend / Full-Stack development experience. Be comfortable reading API docs and writing Python or JavaScript. You do **NOT** need ML math to start as an AI Engineer. If you're missing this — build a simple REST API first. Everything else builds on top of it.

PHASE 01 — FOUNDATION

Understanding the AI Engineering Landscape

Before touching any API, get clear on what role you're actually playing and what tools exist. Most people skip this and spend weeks confused about why things aren't clicking.

01 What is an AI Engineer?

- Builds products using *pre-trained* AI models — not training from scratch
- Think: you use the engine (LLM), you build the car (the product)
- Focus on integration, UX logic, prompt design, and system architecture
- You don't need a PhD. You need good engineering instincts.

✓ ACTION

List 3 AI products you use daily. Think about who built them and how.

02 AI Engineer vs ML Engineer

- **AI Engineer** → uses models via APIs, builds products, focuses on prompts + RAG + agents
- **ML Engineer** → trains models, needs heavy maths, manages data pipelines
- Your job: wire pre-built intelligence into real, useful things
- Impact on product is your north star — not model accuracy metrics

```
✓ ACTION
Search "AI Engineer vs ML Engineer" — read one
article. Know which lane you're in.
```

03 Core Terminology (Memorize These)

- **LLM** — predicts next tokens. That's literally it.
- **Inference** — running the model to get a response
- **Embeddings** — turning text into numbers that capture meaning
- **RAG** — giving the model your own data at query time
- **Agents** — models that can take actions, loop, and plan
- **Context Window** — how much text the model can "see" at once

```
✓ ACTION
Write these 6 terms + definitions on paper.
Close-book style. If you can't, re-read.
```

PHASE 02 — PRE-TRAINED MODELS & APIS

Using Models Without Training Them

Pick a model. Call the API. Get a result. Learn the tradeoffs. Repeat until it's boring — that's when it starts to click.

04

OpenAI Platform Setup

- Go to *platform.openai.com* → create account → get API key
- Explore the **Playground** first — test prompts without writing code
- The **Chat Completions API** is your main tool
- Structure every call: *system role + user message + assistant response*

✓ ACTION

Make your first API call. Print any response to terminal. Don't skip this.

05 Writing Prompts That Work

- Be specific about role, task, format, and constraints
- Use *system prompts* to set persistent behaviour
- Chain prompts for complex tasks — don't cram everything in one shot
- Always test edge cases: what happens with weird/empty/adversarial inputs?

```
✓ ACTION
Go to roadmap.sh/prompt-engineering → read the whole thing once.
```

06 Token Management & Pricing

- ~1 token ≈ 0.75 words (English). Tokens = money.
- **Max tokens** = ceiling of input + output your model handles
- Use *tiktoken* (Python) to count tokens before deploying
- Set output limits in your API call — don't let the model ramble at your cost

```
✓ ACTION
Install tiktoken. Count tokens for 10 prompts. Notice what bloats them.
```

07 Explore the Model Landscape

- **OpenAI** — GPT-4o, most ecosystem support
- **Claude (Anthropic)** — long context, reasoning, writing
- **Gemini (Google)** — multimodal-first, fast
- **Mistral / Cohere** — API or open-weights alternatives
- **Hugging Face** — thousands of open-source models
- Know: context length, knowledge cutoff, and pricing of each

```
✓ ACTION
Run the same prompt on GPT-4, Claude, Gemini.
Compare outputs carefully.
```

08 Fine-tuning (Know When NOT to Use It)

- Fine-tuning = training a model on your custom data to change its behaviour
- **Use it for:** specific tone, output format, domain vocabulary
- **Don't use it for:** adding new knowledge — that's what RAG is for
- It's expensive and slow. Exhaust prompt engineering + RAG first.

```
✓ ACTION
Before fine-tuning anything, ask: "Can a better
system prompt solve this?" Usually yes.
```

09

Open Source Models (Ollama)

- Open source ≠ worse. Mistral 7B beats GPT-3.5 on many tasks.
- **Ollama** — run models locally on your machine (free, private)
- Run: *ollama run llama3* — that's it. Local LLM in 30 seconds.
- Hugging Face Hub = GitHub for models. Filter by task, size, license.
- Transformers.js — run models in the browser

✓ ACTION

Install Ollama. Run llama3 locally. Compare speed + quality vs OpenAI API.

THEN MOVE TO →

PHASE 03 — EMBEDDINGS & VECTOR DATABASES

Making AI Actually Know Your Data

This is where demos turn into actual products. Embeddings let you search by meaning, build recommendation logic, and make the model aware of your own data.

10 What Are Embeddings?

- Text → numbers (vectors) that capture *meaning*, not just keywords
- "dog" and "puppy" are close in embedding space. "dog" and "rocket" are far.
- This lets you search by **meaning** instead of exact words
- Use cases: semantic search, recommendations, anomaly detection, classification

✓ ACTION

Call OpenAI Embeddings API on 5 sentences. Print the vectors. Notice their shape.

11 OpenAI Embeddings API

- Model: *text-embedding-3-small* (cheap, fast, good)
- Send text → get back a 1536-dimension float array
- Compare vectors with **cosine similarity** to find related content
- Pricing is very low — embed large datasets without breaking budget

```
✓ ACTION
Embed 20 sentences. Find the 3 most similar to a query using cosine similarity. Pure Python.
```

12 Vector Databases

- Store millions of embeddings and query them at millisecond speed
- **Chroma** — local, free, great for dev/prototyping
- **Pinecone** — managed, scalable, production-ready
- **Weaviate / Qdrant / FAISS / LanceDB** — alternatives, each with tradeoffs
- Supabase + pgvector = Postgres with vectors (if you want SQL familiarity)

```
✓ ACTION
Pick ONE: Start with Chroma locally. Index 100 docs. Run similarity search.
```

THEN MOVE TO →

PHASE 04 — RAG (RETRIEVAL-AUGMENTED GENERATION)

Teaching the Model Your Own Knowledge

This is how you make a model answer questions about your docs, your data, your product. It's probably the most commonly shipped pattern in AI Engineering right now.

13

How RAG Works

- **Step 1 — Chunk:** split your docs into small pieces (para / page)
- **Step 2 — Embed:** turn each chunk into a vector
- **Step 3 — Store:** put vectors in a vector DB
- **Step 4 — Retrieve:** on query, find the most relevant chunks
- **Step 5 — Generate:** inject chunks into the LLM prompt as context

✓ ACTION

Draw this pipeline on paper. Explain it out loud to someone. It should take 60 seconds.

14 RAG vs Fine-tuning

- **RAG** → add knowledge at query time. Fast, updatable, cheaper.
- **Fine-tuning** → bake knowledge into weights. Slow, expensive, static.
- Rule: if the data changes frequently → use RAG
- Rule: if you need a specific output style/tone consistently → fine-tune
- Most products need RAG, not fine-tuning. Start there.

```
✓ ACTION
Describe your use case. Apply this decision
rule. Lock in your approach.
```

15 Implementing RAG

- **LangChain** — most popular RAG framework. Lots of integrations.
- **LlamaIndex** — data-first approach, better for complex doc pipelines
- **OpenAI Assistant API** — managed RAG. Easiest to ship fast.
- **DIY with SDKs** — most control, most learning. Do this first.

```
✓ ACTION
Build a RAG app from scratch (no framework).
Then use LangChain. Feel the difference.
```

PHASE 05 — AI AGENTS

Models That Can Act, Not Just Talk

Instead of answering once, agents can loop, plan, use tools, and complete multi-step tasks on their own. This is where things get weird and fun.

16

What is an AI Agent?

- A model that can **decide what to do**, do it, check results, and loop
- Has access to *tools*: web search, code execution, APIs, databases
- Uses **ReAct prompting**: Reason → Act → Observe → Repeat
- Can run RAG internally as one of its tools
- Use cases: research assistants, coding agents, workflow automation

✓ **ACTION**

Read the ReAct paper (2022). It's short. It explains 80% of how agents work.

17 Building Agents

- **OpenAI Functions / Tools** — define tools in JSON schema, model decides when to call them
- **OpenAI Assistant API** — managed agents with memory + tools. Easiest start.
- **Manual implementation** — full control, build the loop yourself
- Always add: timeout limits, fallback logic, cost monitoring

✓ ACTION

Build a 2-tool agent: web search + calculator. Give it a math question that needs both.

18 Agent Frameworks

- **LangChain Agents** — widest ecosystem, many pre-built tools
- **LangGraph** — stateful, graph-based agent flows (great for complex logic)
- **LlamaIndex Agents** — data-heavy use cases
- Don't start with a framework — understand the raw loop first
- Frameworks hide complexity that you need to debug in production

✓ ACTION

Build the same agent manually, then in LangChain. Know what the framework is doing for you.

PHASE 06 — MULTIMODAL AI

Beyond Text: Images, Audio, Video

Text is just the starting point. Vision, speech, and image generation APIs are where some of the most interesting products are being built right now — and they're easier to access than you'd think.

19

Image Understanding (Vision API)

- Send an image + text prompt → model describes, analyses, or answers questions about it
- **OpenAI Vision API** — GPT-4o natively sees images
- Use cases: document parsing, product tagging, accessibility tools, UI testing
- Watch out for: hallucinated details in complex images, cost per image

✓ ACTION

Send a photo of a receipt to Vision API. Ask it to extract items + prices as JSON.

20 Image Generation (DALL-E API)

- Text prompt → generated image via DALL-E 3
- Control: size (1024x1024 etc), quality (standard/hd), style (vivid/natural)
- Use cases: content generation, product mockups, marketing assets
- Hugging Face hosts Stable Diffusion + many other image models open-source

✓ ACTION
Generate 5 images with varied prompts. Note what makes prompts produce better results.

21 Audio: Whisper (Speech-to-Text) + TTS

- **Whisper API** — transcribe audio files or real-time speech. Insanely accurate.
- Supports 50+ languages. Great for meeting notes, subtitles, voice commands.
- **TTS API** — convert text to natural-sounding speech
- Open source: run WhisperX locally on GPU for batched, fast transcription

✓ ACTION
Record a 60-sec voice memo. Transcribe it with Whisper API. Check accuracy.

THEN MOVE TO →

PHASE 07 — AI SAFETY, ETHICS & PRODUCTION

Shipping AI Responsibly

Not optional. If you're shipping AI products, you're responsible for what they do. Learn the failure modes and build guards in from the start — retrofitting safety is a nightmare.

22

Prompt Injection & Security

- **Prompt injection** = user input that hijacks your system prompt
- Example: user types "Ignore all previous instructions and..." — your agent obeys
- Defences: input sanitisation, separate trusted/untrusted channels, output validation
- Never trust model output as ground truth — always validate

✓ ACTION

Try to break your own app with a prompt injection. If it works, fix it before shipping.

23 Bias, Fairness & Privacy

- LLMs reflect training data biases — they exist, know it
- Test your app with diverse inputs. Look for unfair or harmful outputs.
- **Privacy:** don't send PII to external APIs unless you've checked their data policy
- Add end-user IDs in API calls for monitoring + abuse detection

```
✓ ACTION
Read OpenAI's usage policies. Know what you're
not allowed to build.
```

24 Moderation & Output Constraints

- **OpenAI Moderation API** — free classifier for harmful content. Use it.
- Constrain inputs: limit what topics users can ask about in your system prompt
- Constrain outputs: specify format, length, and what to refuse in your prompt
- Adversarial testing: actively try to break your own system before users do

```
✓ ACTION
Add the Moderation API to any app you ship.
Costs almost nothing. Prevents a lot.
```

// NOW THE FUN PART — Things to Add in your Resume

FRAMEWORKS

LangChain, LlamaIndex, LangGraph, OpenAI SDK, Anthropic SDK

VECTOR DBS

Chroma (dev), Pinecone (prod), Qdrant, FAISS, LanceDB, Supabase

OPEN SOURCE

Ollama, Hugging Face Hub, Sentence Transformers, Transformers.js

CLOUD PLATFORMS

Azure AI, AWS SageMaker, Replicate, Together.ai, Groq

MODELS TO KNOW

GPT-4o, Claude 3.5+, Gemini 1.5+, Llama 3, Mistral 7B

EMBEDDING MODELS

text-embedding-3-small (OAI), all-MiniLM (SBERT), e5-large

AUDIO / VISION

Whisper API, WhisperX (local), DALL-E 3, GPT-4o Vision

SAFETY TOOLS

OpenAI Moderation API, tiktoken, Guardrails AI, Langfuse

@bunnybethinking

AI is not that hard. We're just learning it wrong.